

HireMind Implementation & Architectural Report

This document serves as the comprehensive implementation and architectural report for the **HireMind** Full-Stack Resume Matching and Recruiter Summarization platform. It outlines the structural design, data workflows, algorithms, and architectural layers implemented across the frontend and backend applications.

Full-Stack Specifications & Data Model

1. System Overview & Architecture

HireMind is engineered as an offline-first, local-privacy-centric recruiting application. The system allows human resources personnel to load resumes (in PDF/DOCX format) and job requirements, match skills automatically, and generate deep evaluation summaries.

The system is split into:

- **FastAPI Backend:** A lightweight, async-first Python server communicating via JSON endpoints. It coordinates document processing, regular expression/NLP-based skill extraction, database indexing, and RAG compilation.
 - **Vite/React Frontend:** A premium, responsive interface featuring interactive metrics dashboards, drag-and-drop file uploaders, and a recruiter evaluation summary dashboard.
 - **SQLite Database:** Local SQLite file storage managed via async SQLAlchemy ORM.
-

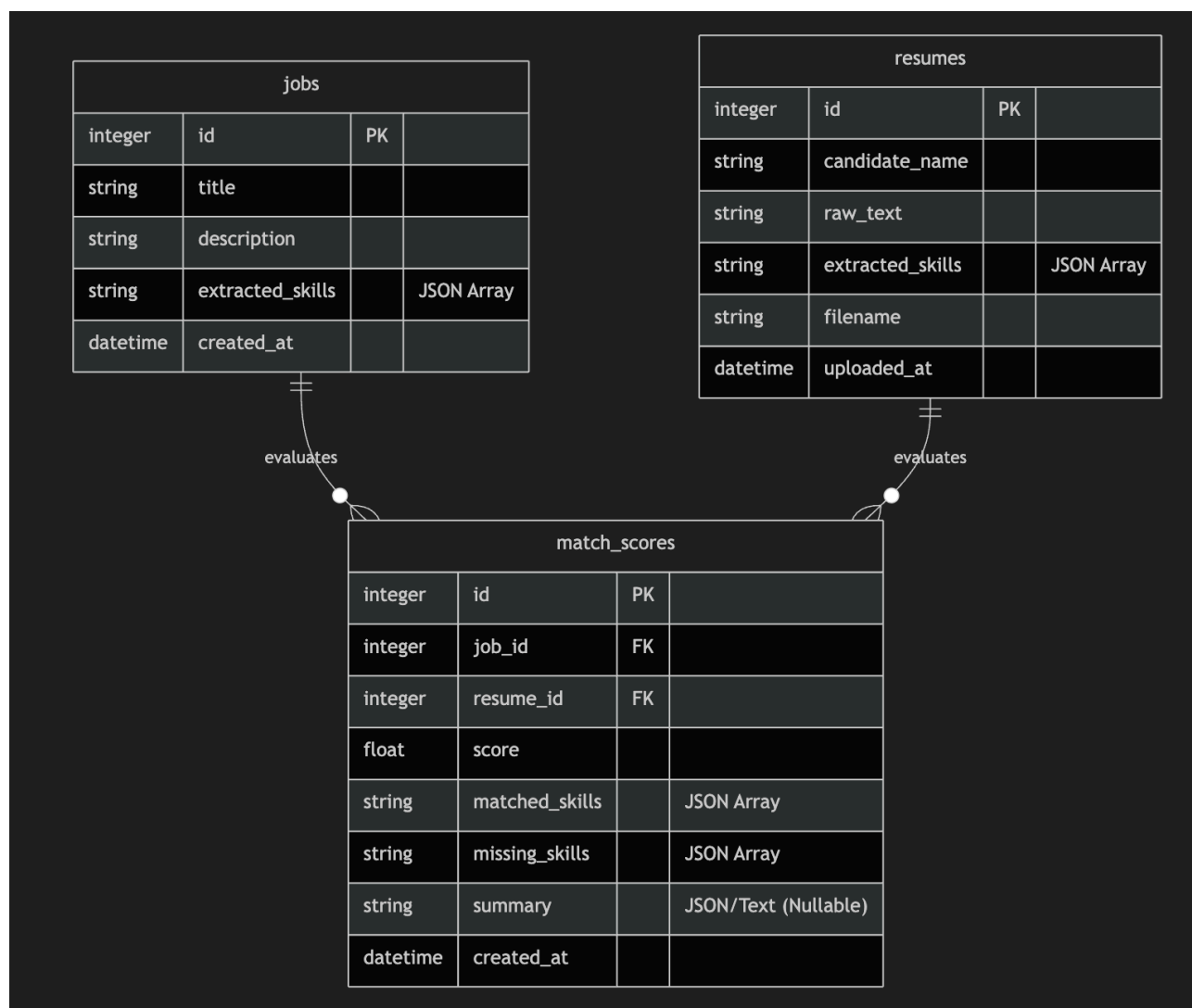
2. Database Models & Dynamic Migrations

The persistence layer is defined in `backend/app/database.py` with three core tables: Job, Resume, and MatchScore.

Dynamic Safe Migration Engine

To support incremental upgrades without losing historical records, the database startup routine (`init_db`) checks the table metadata dynamically. It attempts to migrate existing tables rather than dropping them:

- Queries SQLite's `PRAGMA table_info("match_scores")`.
 - If the `summary` column is missing, it dynamically runs `ALTER TABLE match_scores ADD COLUMN summary TEXT` to append the column safely while maintaining database integrity.
-



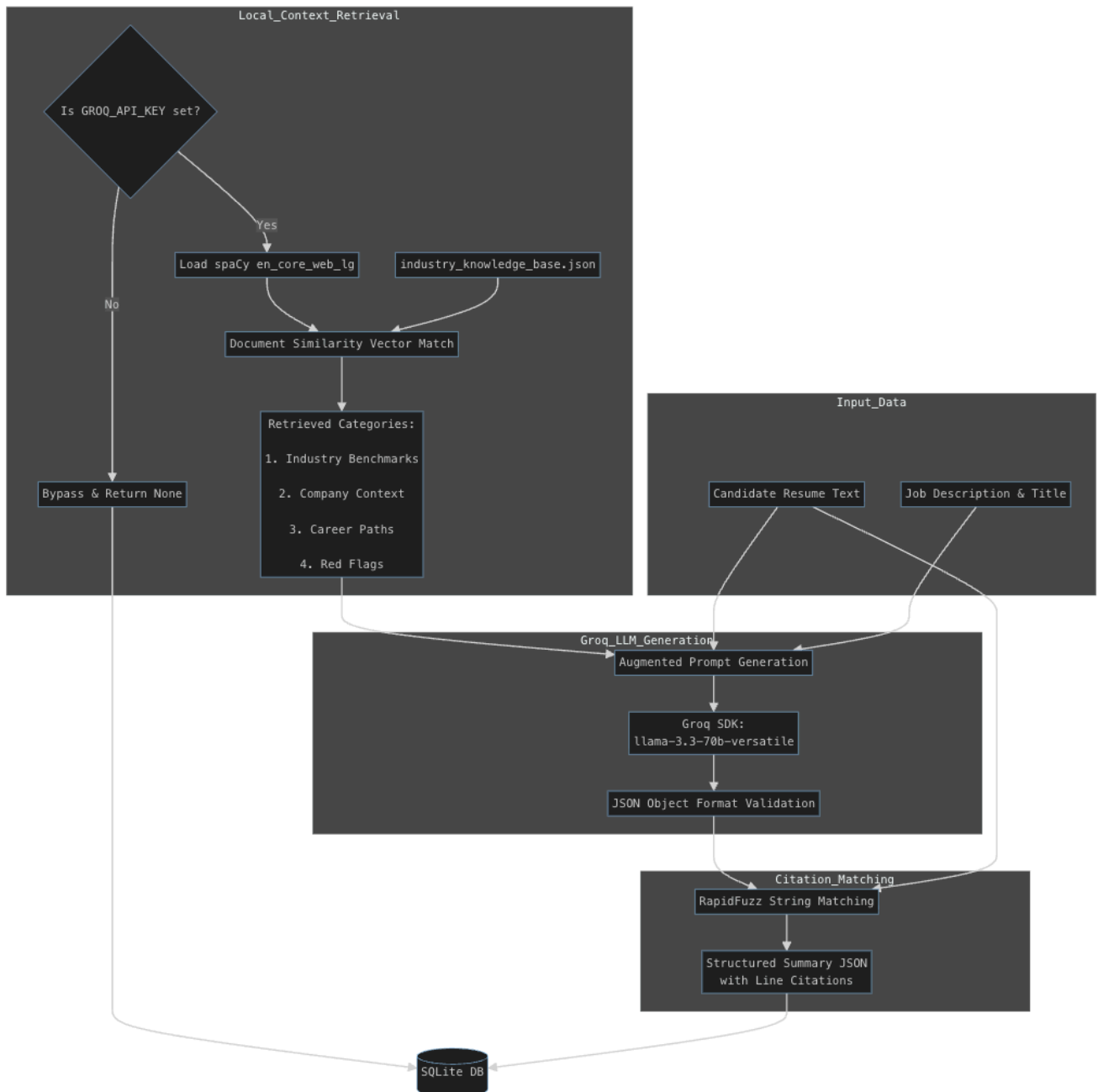
3. Core Matching & Skill Extraction Algorithms

- **Skill Extraction:** Implemented in [skill_extractor.py](#). When a file is parsed, the text is scanned using compiled case-insensitive regular expression boundaries against a catalog of 200+ industry-standard technologies (skills.json).
- **Matching Engine:** Implemented in [matcher.py](#). It calculates match percentages using asymmetric set comparison:
$$\text{Score} = \left(\frac{|\text{Matched Skills}|}{|\text{Job Required Skills}|} \right) \times 100$$
 Fuzzy string matching via a configurable Levenshtein Distance threshold is used to prevent spelling or synonym mismatches.

RAG-based Recruiter Summarizer & Citations

1. RAG Summarizer Architecture

The recruiter summary feature is implemented in [summarizer.py](#). It operates as a Retrieval-Augmented Generation (RAG) agent that extracts and reviews candidate alignment details.



2. RAG Execution Breakdown

A. Resource Optimization & Lazy Initialization

To keep the application startup rapid and light, spaCy model loading (`spacy.load("en_core_web_lg")`) and knowledge base reading are deferred. If `GROQ_API_KEY` is not present in `.env` (or is empty), the client is disabled:

- The spaCy model is not loaded (saving ~800MB RAM).
- Any summary request immediately returns `None` without raising runtime exceptions, keeping the primary scoring endpoint robust.

B. Context Retrieval

When the API key is active, spaCy document vector representation measures the cosine similarity between the target role and the knowledge base keywords. The system pulls the top 2-3 most relevant standards from:

- **industry_benchmarks:** Expected libraries, coding standards, and tooling.
- **career_path:** Seniority levels, architecture ownership, and scale benchmarks.
- **company_context:** Cultural items, security benchmarks, code reviews, and remote collaboration.
- **red_flags:** Unexplained gaps, repeating job-hopping, and keyword stuffing.

C. Groq LLM Inference & JSON Schemas

The context-augmented payload is sent to Groq's llama-3.3-70b-versatile model. We set `response_format={"type": "json_object"}` to guarantee response parsing compatibility. The model outputs:

1. **Executive Summary:** A high-level overview text with a confidence indicator (HIGH | MEDIUM | LOW).
2. **Skills Alignment Matrix:** An array of objects with the skill name, candidate level, gap severity, and gap mitigation plan.
3. **Potential Concerns:** Flagged items and mitigating candidate experience metrics.
4. **Recruiter Next Steps:** Structured check questions and interview priorities.

D. RapidFuzz Citation Anchoring

To prevent hallucinated evaluations, the backend validates the LLM's evidence statements against the original resume using the **RapidFuzz** library's `partial_ratio` algorithm:

- Iterates through every line in the raw resume.
- Calculates the Levenshtein-based matching ratio.
- If similarity is above a threshold of 65, it appends a precise citation anchor (e.g. Line 18, Experience/Details), which is rendered dynamically in the recruiter dashboard.